

Programação assíncrona

Eventos e tratamento assíncrono

- Utilização do **Event Loop**
- Eventos: data, end, close, error
- Tratamento via função **callback**
- Recepção segmentada (stream)

```
reader.on('data', (chunk) => {  
  console.log(chunk); });
```

- Biblioteca **net**, evento **error**

```
server.on('error', (e) => {
  if (e.code == 'EADDRINUSE') {
    console.log('Endereço em uso, nova
tentativa...');
    setTimeout( ( ) => {
      server.close(); server.listen(PORT, HOST);
    }, 1000);
  });
});
```

```
const http = require('http');

const server = http.createServer((req,res)
=>{
  res.write("A");
  res.write("B");
  res.end("C");
}).on("close",()=>console.log("Servidor
encerrado"));

setTimeout(()=>server.close(),3000);
server.listen(3000);
```

```
const req = http.request(  
  {hostname: 'localhost', port: 3000,  
   method: 'GET'},  
  (res) => {  
    res.on('data', (d) => console.log("Resp:  
"+d.toString()));  
    res.on('end', () => console.log("Fim da  
comunicação"));  
    res.on('close', () => console.log("Conexão  
encerrada"));  
  });  
req.end();
```

```
Resp: A  
Resp: B  
Resp: C  
Fim da comunicação  
Conexão encerrada  
Servidor encerrado
```